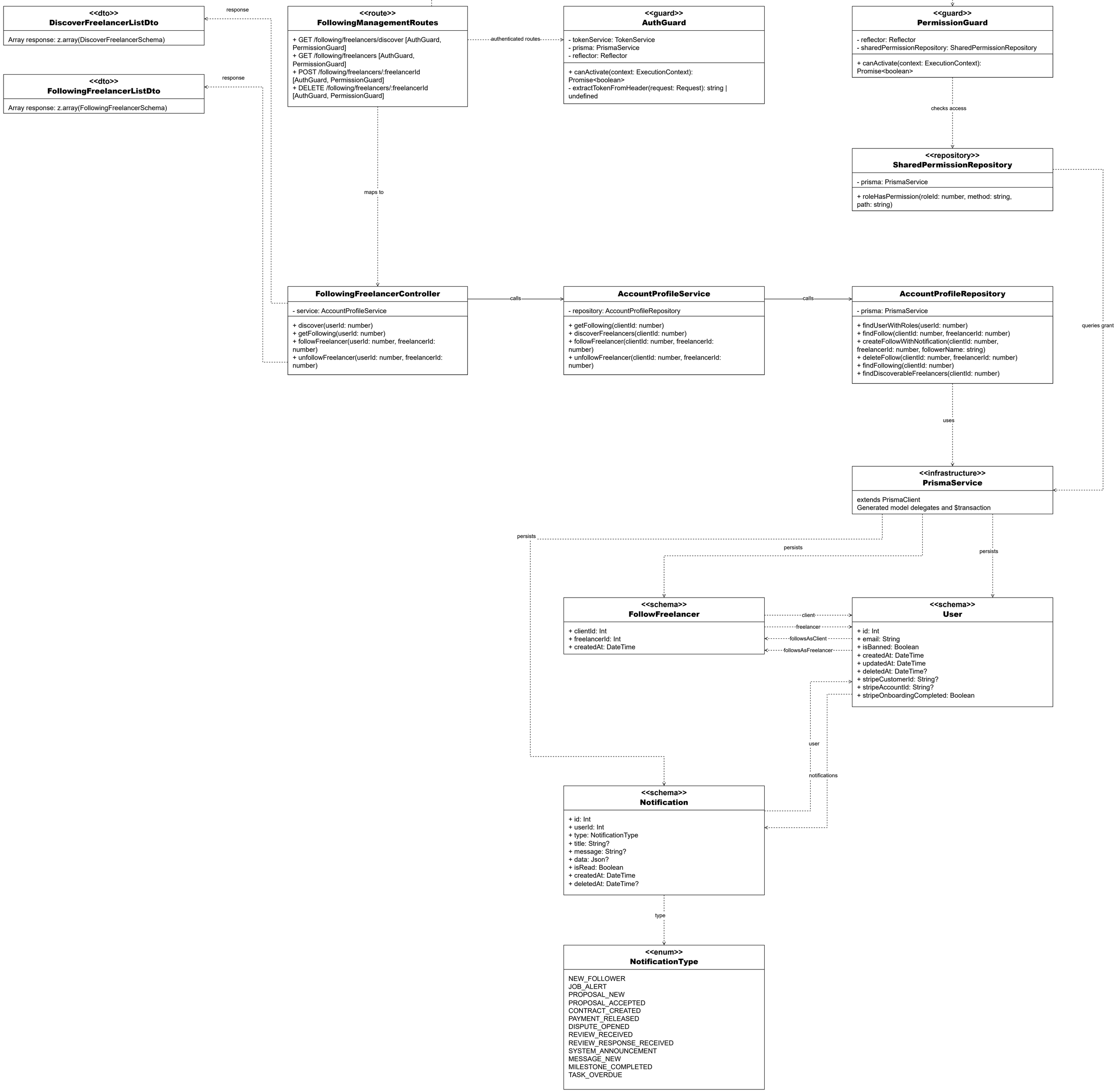

authenticated routes
<<dto>>
DiscoverFreelancerListDto
Array response: z.array(DiscoverFreelancerSchema)
response
<<route>>
FollowingManagementRoutes
+ GET /following/freelancers/discover [AuthGuard, PermissionGuard]
+ GET /following/freelancers [AuthGuard, PermissionGuard]
+ POST /following/freelancers/:freelancerId [AuthGuard, PermissionGuard]
+ DELETE /following/freelancers/:freelancerId [AuthGuard, PermissionGuard]
authenticated routes
<<guard>>
AuthGuard
- tokenService: TokenService
- prisma: PrismaService
- reflector: Reflector
+ canActivate(context: ExecutionContext): Promise<boolean>
- extractTokenFromHeader(request: Request): string | undefined
<<guard>>
PermissionGuard
- reflector: Reflector
- sharedPermissionRepository: SharedPermissionRepository
+ canActivate(context: ExecutionContext): Promise<boolean>
checks access
<<dto>>
FollowingFreelancerListDto
Array response: z.array(FollowingFreelancerSchema)
response
maps to
<<repository>>
SharedPermissionRepository
- prisma: PrismaService
+ roleHasPermission(roleId: number, method: string, path: string)
FollowingFreelancerController
- service: AccountProfileService
+ discover(userId: number)
+ getFollowing(userId: number)
+ followFreelancer(userId: number, freelancerId: number)
+ unfollowFreelancer(userId: number, freelancerId: number)
calls
AccountProfileService
- repository: AccountProfileRepository
+ getFollowing(clientId: number)
+ discoverFreelancers(clientId: number)
+ followFreelancer(clientId: number, freelancerId: number)
+ unfollowFreelancer(clientId: number, freelancerId: number)
calls
AccountProfileRepository
- prisma: PrismaService
+ findUserWithRoles(userId: number)
+ findFollow(clientId: number, freelancerId: number)
+ createFollowWithNotification(clientId: number, freelancerId: number, followerName: string)
+ deleteFollow(clientId: number, freelancerId: number)
+ findFollowing(clientId: number)
+ findDiscoverableFreelancers(clientId: number)
queries grants
uses
<<infrastructure>>
PrismaService
extends PrismaClient
Generated model delegates and $transaction
persists
persists
persists
<<schema>>
FollowFreelancer
+ clientId: Int
+ freelancerId: Int
+ createdAt: DateTime
client
freelancer
followsAsClient
followsAsFreelancer
<<schema>>
User
+ id: Int
+ email: String
+ isBanned: Boolean
+ createdAt: DateTime
+ updatedAt: DateTime
+ deletedAt: DateTime?
+ stripeCustomerId: String?
+ stripeAccountId: String?
+ stripeOnboardingCompleted: Boolean
user
notifications
<<schema>>
Notification
+ id: Int
+ userId: Int
+ type: NotificationType
+ title: String?
+ message: String?
+ data: Json?
+ isRead: Boolean
+ createdAt: DateTime
+ deletedAt: DateTime?
type
<<enum>>
NotificationType
NEW_FOLLOWER
JOB_ALERT
PROPOSAL_NEW
PROPOSAL_ACCEPTED
CONTRACT_CREATED
PAYMENT_RELEASED
DISPUTE_OPENED
REVIEW_RECEIVED
REVIEW_RESPONSE_RECEIVED
SYSTEM_ANNOUNCEMENT
MESSAGE_NEW
MILESTONE_COMPLETED
TASK_OVERDUE